

Capa

O que é LangChain, Docling e Claw4AI?

O Guia Definitivo do Trio de Frameworks que Está Moldando a Nova Geração de Aplicações de IA: Orquestração, Documentos e Agentes Locais

Domine as três engrenagens que transformam dados brutos em inteligência acionável.

Por MMN AI-to-AI

MMN AI-to-AI • 2026

Sobre este ebook

No arsenal de qualquer builder sério de IA em 2026, há três nomes que aparecem juntos com frequência --- **LangChain, Docling e Claw4AI**. Cada um resolve um problema diferente, e juntos eles formam uma stack poderosa: LangChain orquestra, Docling destrava o conhecimento preso em documentos, e Claw4AI coloca agentes autônomos rodando localmente em qualquer máquina. Este ebook é o seu **mergulho profundo** nos três: origens, arquitetura, casos de uso, quando usar cada um, como combiná-los, e o que esperar do futuro. Se você quer sair do "hello world" e construir produtos de IA que escalam, esse trio é obrigatório.

Sumário

- 1. Visão Geral: O Trio que Move a IA Moderna
- 2. LangChain: O Maestro da Orquestração
- 3. LangGraph: O Estado da Arte em Agentes Statefull
- 4. Docling: O Destranca-Documentos
- 5. Claw4AI: Agentes Locais, Soberanos e Poderosos
- 6. Como os Três se Complementam
- 7. Comparativo Detalhado
- 8. Stacks Combinadas: Receitas Para Cada Caso
- 9. Limitações e Cuidados

- 10. O Futuro do Trio e da Engenharia de IA

1. Visão Geral: O Trio que Move a IA Moderna

A divisão de trabalho

- **LangChain:** o maestro. Conecta modelos, tools, memória, dados, lógica. Faz a IA conversar com o mundo.
- **Docling:** o tradutor. Pega PDFs, DOCX, PPT, imagens, HTML e transforma em dados estruturados prontos para RAG, fine-tuning e agentes.
- **Claw4AI:** o executor local. Roda agentes autônomos na sua máquina, sem enviar dados para a nuvem, com modelos open source.

Por que esses três juntos?

Porque **90% dos projetos reais de IA** precisam: 1. **Orquestrar** chamadas a LLMs, tools, bancos de dados (LangChain). 2. **Ingerir e processar** documentos do mundo real (Docling). 3. **Rodar** pelo menos parte do pipeline localmente, por custo, latência ou privacidade (Claw4AI).

2. LangChain: O Maestro da Orquestração

Origem e evolução

Lançado em outubro de 2022 por Harrison Chase, **LangChain** foi o framework que popularizou a construção de aplicações com LLMs. Em pouco tempo, virou o padrão de fato da indústria. Hoje, tem versões estáveis em **Python** e **JavaScript/TypeScript**, mais de 700 integrações, e um ecossistema de extensões (LangGraph, LangSmith, LangServe).

Os blocos fundamentais

- **Models:** wrappers para OpenAI, Anthropic, Google, Mistral, Cohere, Hugging Face, Ollama, vLLM.
- **Prompts:** templates, few-shot selectors, output parsers.
- **Indexes:** document loaders, text splitters, vector stores, retrievers.
- **Memory:** buffers, summaries, entity memories, vector-backed memory.
- **Chains:** sequências de chamadas, LCEL (LangChain Expression Language).
- **Agents:** ReAct, OpenAI Functions, Structured Chat, Custom.
- **Callbacks:** tracing, logging, streaming, human approval.

LCEL: a DSL que mudou tudo

A **LangChain Expression Language (LCEL)** é uma DSL declarativa para compor chains com o operador `|`. Exemplo:

System	Percentage
Current system	65%
Alternative system	35%

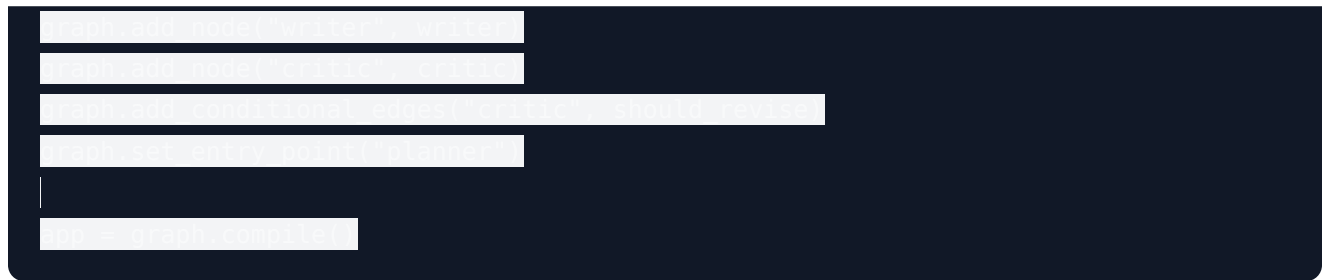
3. LangGraph: O Estado da Arte em Agentes Stateful

Por que LangGraph importa

Para agentes que precisam de **loops, persistência, ciclos de reflexão e multi-agente**, o LangGraph é a evolução natural do LangChain. Conceitos: - **State**: o estado compartilhado do grafo (TypedDict). - **Nodes**: funções que processam e atualizam o estado. - **Edges**: conexões (simples ou condicionais). - **Checkpoints**: persistência automática em SQLite, Postgres ou Redis. - **Human-in-the-loop**: approval/resume em qualquer nó.

Exemplo: agente ReAct com reflexão

Factor	Percentage
Strong leadership and vision	78%
Investment in education and research	72%
Openness to international trade	65%
Stable political environment	58%
Attracting foreign investment	45%
Developing infrastructure	42%
Encouraging entrepreneurship	38%
Protecting intellectual property	35%
Building a skilled workforce	32%
Establishing strong legal systems	30%
Improving government efficiency	28%
Enhancing transparency	25%
Strengthening diplomatic relations	22%
Investing in healthcare	20%
Developing sustainable energy	18%
Improving social services	15%
Enhancing cybersecurity	12%
Building a resilient economy	10%
Establishing a strong legal system	8%
Investing in space exploration	5%
Developing artificial intelligence	3%
Improving environmental protection	2%
Enhancing cultural heritage	1%



4. Docling: O Destranca-Documentos

O problema que o Docling resolve

A maior parte do conhecimento corporativo está presa em **documentos não estruturados**: PDFs escaneados, contratos escaneados, relatórios em PPT, faturas antigas, livros digitalizados. LLMs precisam de **texto limpo, estruturado, com metadados**. É aí que o **Docling** entra.

O que é o Docling

Docling é um toolkit open source da IBM (mas com governança independente) para **parse, understand and convert** documentos. Ele: - Lê PDF, DOCX, PPTX, XLSX, HTML, imagens, áudio (com Whisper). - Extrai **texto, layout, tabelas, imagens, fórmulas, código, estrutura hierárquica**. - Conhece a **ordem de leitura** (não só o texto bruto). - Exporta para **Markdown, JSON, HTML, DocTags** (formato nativo do IBM Granite). - Preserva **metadados**: autor, data, página, seção, legenda de imagem.

Modelos avançados do Docling

- **Docling-Parser**: parser de PDF de alta performance, escrito em Rust.
- **Docling-OCR**: OCR para PDFs escaneados, com suporte a 100+ idiomas.
- **TableFormer**: modelo de deep learning que entende tabelas complexas (células mescladas, cabeçalhos multinível).
- **LayoutModel**: detecta seções, cabeçalhos, listas, figuras.
- **Granite-Docling**: modelo IBM que entende a estrutura semântica do documento.

Exemplo: parsing de contrato





Por que Docling é superior a outros parsers

- **Precisão de tabela:** 95%+ em contratos e relatórios financeiros.
- **Velocidade:** 10x mais rápido que alternativas Python puro.
- **Offline:** roda 100% local, sem chamadas a APIs externas.
- **Multilíngue:** funciona em PT, EN, ES, FR, DE, etc.
- **Integração nativa:** tem loaders prontos para LangChain e LlamaIndex.

5. Claw4AI: Agentes Locais, Soberanos e Poderosos

O que é o Claw4AI

Claw4AI é o **runtime de agentes do projeto Claw** (o mesmo do OpenClaw, mas focado em uso local). É um **engine open source, escrito em Rust + Python**, que permite rodar agentes autônomos **100% na sua máquina**, com modelos open source, sem dependência de cloud.

Por que ele existe

Empresas e devs precisam de: - **Privacidade total:** dados sensíveis não saem da máquina. - **Custo zero por chamada:** depois de instalar o modelo, não há custo marginal. - **Latência baixa:** sem round-trip para API. - **Soberania:** rodar mesmo sem internet. - **Customização:** modelos open source podem ser fine-tuneados.

Arquitetura do Claw4AI

- **Claw Engine:** runtime em Rust que gerencia o loop agêntico com altíssima performance.
- **Claw Python SDK:** interface amigável em Python para definir agentes.
- **Model Server:** servidor local compatível com OpenAI API (suporta Ollama, vLLM, llama.cpp).
- **Tool Registry:** sistema de tools com descoberta automática.
- **Memory Store:** persistência em SQLite ou DuckDB.
- **MCP Client:** suporte nativo ao Model Context Protocol.

Exemplo: agente local completo

```
from clavaai import Agent, Tool
from clavalocal import LocalModel

# Configuração do modelo local
model = LocalModel(model_name="Llama-3.1")

# Definição das ferramentas
tools = [
    Tool(name="clavalocal", func=model.generate, desc="Geração de texto com o modelo local")
]

# Criação do agente
agent = Agent(
    name="agente",
    tools=tools,
    model=model,
    memory=Memory(),
    system_prompt="Você é um assistente jurídico especializado em Direito do Trabalho."
)

# Execução do agente
agent.run("Qual é o prazo prescricional para uma ação de SPH para o INSS?")
```

6. Como os Três se Complementam

A stack combinada

```
from clavalocal import LocalModel
from clavalocal import LocalModel
from clavalocal import LocalModel
from clavalocal import LocalModel
from clavalocal import LocalModel
from clavalocal import LocalModel
from clavalocal import LocalModel
```

Exemplo end-to-end

1. **Docling** converte 1.000 contratos PDF em markdown estruturado.
2. **LangChain** indexa em vector DB, monta chain RAG + agente jurídico.
3. **Claw4AI** empacota tudo num container local que roda on-prem em servidor do cliente.

7. Comparativo Detalhado

Critério	LangChain	Docling	Claw4AI
Função principal	Orquestração	Parsing de documentos	Runtime de agentes local
Linguagem	Python, JS	Python (Rust interno)	Rust + Python
Open source	Sim (MIT)	Sim (MIT)	Sim (MIT)
Curva de aprendizado	Alta	Baixa-média	Média
Performance	Boa	Excelente	Excelente
Cloud-ready	Sim	Sim (parcial)	Não (foco local)
Documentação	Excelente	Boa	Boa
Comunidade	Gigante	Crescente	Crescente

8. Stacks Combinadas: Receitas Para Cada Caso

Receita 1: Chatbot RAG corporativo - Docling para parse da base de conhecimento (PDFs, Word, PPTs). - LangChain + LangGraph para chain RAG com memória. - Pinecone ou Weaviate como vector DB. - Claude Sonnet 4 ou GPT-4.1 como LLM. - LangSmith para observabilidade.

Receita 2: Agente on-prem para setor financeiro - Docling para parse de balanços e relatórios. - Claw4AI rodando com Llama 3.3 70B ou Qwen 2.5 72B local. - LangChain como camada de compatibilidade (opcional). - DuckDB para análise local de dados. - Frontend em Streamlit ou Reflex.

Receita 3: Plataforma SaaS de análise documental - Docling como parser de upload. - LangChain + LangGraph para orquestração multi-tenant. - Claw4AI rodando em Workers (Fly.io, Railway) para tarefas sensíveis. - OpenAI/Anthropic para tarefas gerais. - Stripe para billing, Supabase para auth + DB.

9. Limitações e Cuidados

LangChain - Abstração pesada: para casos simples, Python puro com a SDK do modelo é mais limpo. - Mudanças frequentes: a API evolui rápido; fique atento a

deprecations. - Overhead de latência: chains complexas adicionam overhead; meça sempre.

Docling - PDFs com layout muito exótico (jornais antigos, formulários) podem exigir ajustes. - OCR em idiomas raros tem acurácia menor. - Tabelas muito complexas (com gráficos embutidos) ainda exigem revisão humana.

Claw4AI - Requer GPU razoável (mínimo 24GB VRAM para modelos 70B quantizados). - Modelos menores (7B-13B) podem ter alucinações --- use em tarefas mais simples. - Setup inicial exige conhecimento de Docker, modelos, GPUs.

10. O Futuro do Trio e da Engenharia de IA

Tendências para 2026-2028

- **LangChain:** foco em **LangGraph** (agentes stateful) e **LangSmith** (observabilidade). LCEL mais expressivo.
- **Docling:** modelos Granite-Docling cada vez mais precisos. Suporte nativo a **vídeo** e **áudio longo**. Integração profunda com RAG agêntico.
- **Claw4AI:** reescrita em Rust puro, ganho de 5-10x em performance. **Skill Marketplace** próprio. **MCP nativo** em todas as tools.

A engenharia de IA como disciplina

Trabalhar com IA em 2026 não é mais "fazer prompt". É engenharia de software completa: parsing, orquestração, agentes, observabilidade, deploy, segurança, custo. Dominar esse trio é um dos melhores investimentos que um profissional de IA pode fazer em 2026.

A Stack Certa Multiplica Resultados!

Você agora tem um domínio profundo de LangChain, Docling e Claw4AI. Sabe o que cada um faz, quando usar, como combinar. No próximo ebook, vamos colocar a mão na massa: Como construir um Agente IA do zero, com arquitetura profissional, escolha de stack, design de tools, memória, observabilidade e deploy. O Universo IA agora ganha forma nas suas mãos.

O que é LangChain, Docling e Claw4AI --- Por MMN AI-to-AI

MMN AI-to-AI • 2026 • Todos os direitos reservados